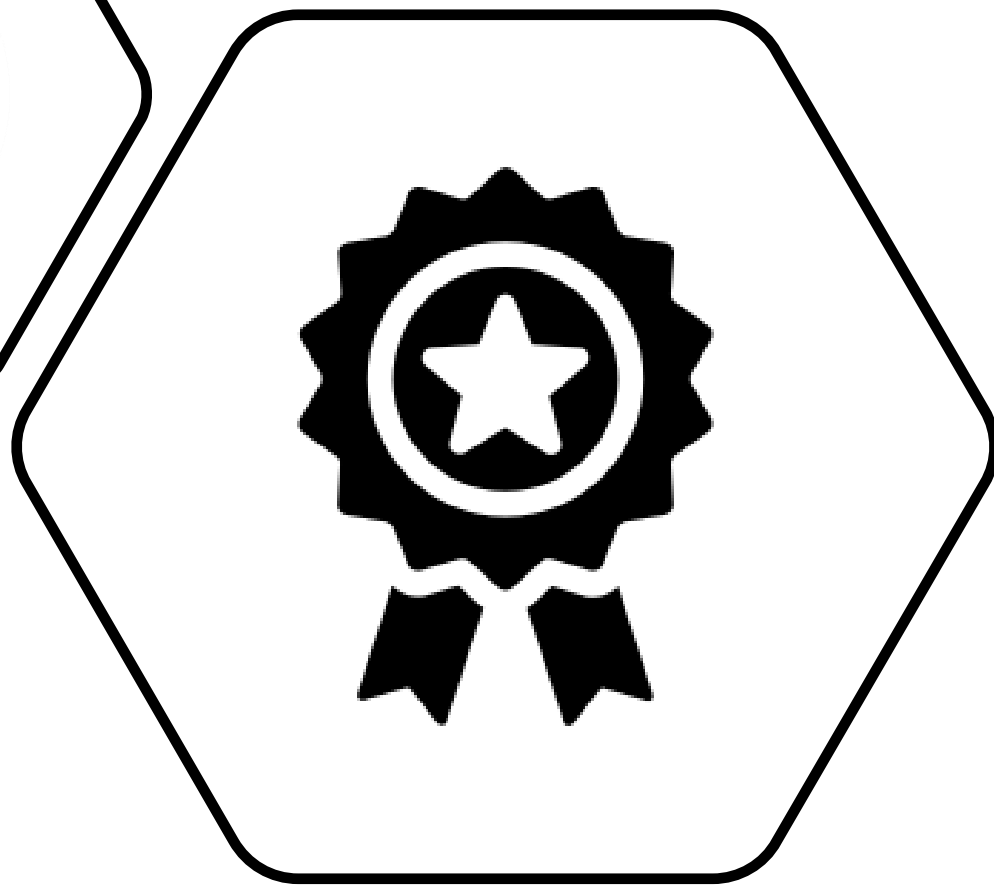
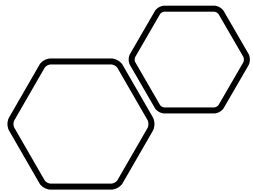
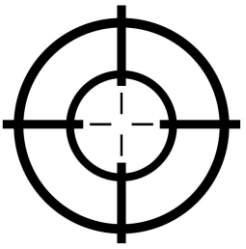




Código limpo e
princípios SOLID

Quais são os principais problemas associados ao desenvolvimento de Software?



Existem vários fatores a se considerar e, sinceramente, talvez enumerar todos não seja possível.

Desenvolver e/ou lançar um novo software pode ser complicado. Isto porque em cada etapa do processo de desenvolvimento e aprovação, existem muitas coisas a serem consideradas.

Problema de integração



Pode ser um desafio integrar o que está se fazendo com ferramentas que seu público/clientes já esteja usando ou ainda integrar ferramentas que se domina com outras que seus público/clientes querem.

As soluções passam por encontrar maneiras de tornar seu produto compatível usando interfaces (application program interfaces - APIs) ou fazendo parceria com proprietários das outras ferramentas.

Falhas na comunicação



O desenvolvimento de software requer uma equipe colaborando para atender às necessidades do cliente. Cada membro deve conhecer a estratégia de codificação, o objetivo e as metas de um projeto. Caso contrário, as consequências afetam a reputação do gerente e a produção da equipe.

Um bom gerente garantirá que cada membro da equipe conheça as múltiplas facetas de um projeto e instrua imediatamente os novos membros. Isso aumentará a sinergia e a produtividade.

Cronogramas irrealistas ou mal gerenciados



Um dos problemas mais comuns no gerenciamento de projetos é a linha do tempo. Ser realista na definição de cronogramas com projetos é fundamental.

Uma das melhores maneiras de evitar atrasos nos cronogramas é fazer um brainstorm no início do projeto.

Sobrecarga de recursos



O dos maiores obstáculos quando se trata de lançar um novo software é o desejo de colocar muito nele.

A solução para superar isso é ser mais especializado em vez de mais geral.

Falta de alinhamento entre as necessidades do “patrão” e do cliente



A opinião do seu “patrão” sobre o que deve ser construído deve ser secundária em relação às necessidades do usuário final. Sejam realistas, as chances de seu chefe ser um visionário, como o Steve Jobs, são muito pequenas.

Eduque-os sobre a importância de permitir que o mercado diga a você quais recursos desenvolver.

Não identificar o verdadeiro “porquê”



O problema mais comum é construir a coisa errada. As empresas precisam entender não apenas o que um cliente está pedindo, mas “porquê” ele está pedindo isso especificamente.

Determinar o que interessa pode garantir a entrega correta.

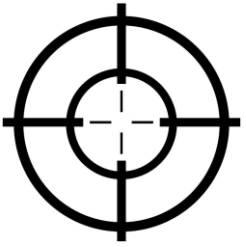
Avanço rápido da tecnologia



Cada avanço tecnológico é importante para a indústria de TI. Mas, ao mesmo tempo, a evolução da tecnologia a uma taxa fenomenal leva a uma pressão adicional para os profissionais de desenvolvimento de software.

Esta pressão é, geralmente, para alavancar essas tendências de tecnologia no desenvolvimento de produtos de software, para obter uma vantagem sobre os concorrentes e/ou se destacar no mercado.

O que fazer então?

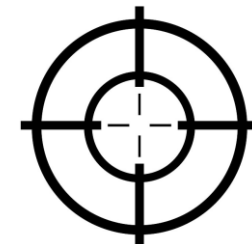


Resolver todos estes problemas não é possível. Mas é possível aumentar a capacidade técnica dos desenvolvedores.

Aumentar a capacidade técnica pode “equilibrar as coisas”. Assim é possível evitar ou diminuir o impacto provocado por algum desses problemas.

Deve-se seguir princípios e/ou boas práticas de desenvolvimento.

Código limpo



Mesmo um código ruim pode funcionar. Mas se ele não for limpo, pode acabar com uma empresa de desenvolvimento. Perdem-se a cada ano horas incontáveis e recursos importantes devido a um código mal escrito.

Alguém já perdeu tempo tentando entender algum código, feito por outra pessoa.

O que é um código limpo?



Código limpo é um código fácil de entender e alterar.

Fácil de entender significa que o código é fácil de ler, seja o leitor o autor original do código ou outra pessoa. Seu significado é claro, portanto, minimiza a necessidade de suposições e a possibilidade de mal-entendidos.

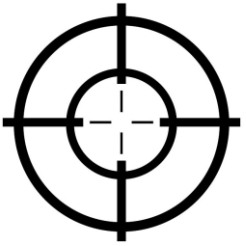
Para tanto deve-se seguir os princípios de um código limpo...

Clareza

Acima de tudo, código limpo tem que ter clareza

Fácil de entender o fluxo de execução de todo o aplicativo. Fácil de entender como os diferentes objetos colaboram entre si. Fácil de entender o papel e a responsabilidade de cada classe. Fácil de entender o que cada método faz. Fácil de entender qual é o propósito de cada expressão e variável.

Nomes significativos



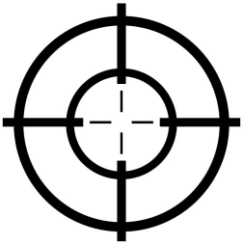
O que este código faz? O que será calculado?

```
const calc = (p, l) => {  
    return p / (l * l)  
}
```



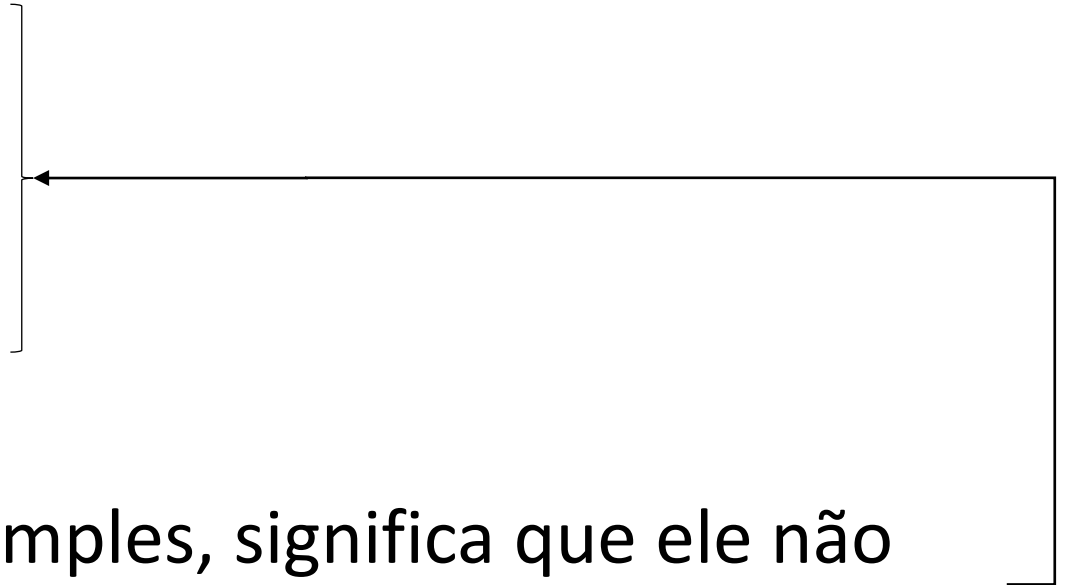
É possível dizer os tipos de cada parâmetro, mas não o que as variáveis significam. Sem saber o que as variáveis significam, pode-se perder o entendimento do contexto onde o código deve/pode ser aplicado.

Nomes significativos



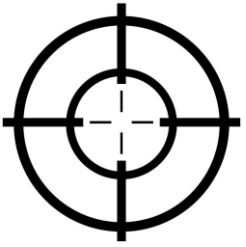
E agora que está comentado, é possível entender o que o código faz?

```
// Cálculo do índice de massa corporal  
// p é o peso, l é a altura  
const calc = (p, l) => {  
    return p / (l * l)  
}
```

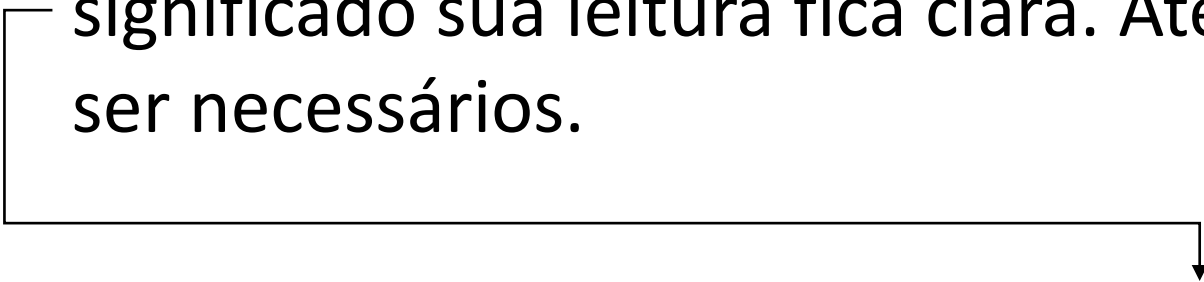


Se é necessário comentar um código simples, significa que ele não está simples. As variáveis não possuem nomes significativos.

Nomes significativos



Quando a nomenclatura de variáveis, métodos e objetos tem significado sua leitura fica clara. Até mesmo comentários deixam de ser necessários.

A line with a downward-pointing arrow originates from the word "significado" in the text above and points to the variable name in the code below.

```
const indiceMassaCorporal = (peso, altura) => {  
  return peso / (altura * altura)  
}
```

Refazendo a pergunta. O que este código faz?

Como nomear classes e métodos?



Prefira sempre colocar o nome de classes como substantivos, com significado. Métodos com nomes de verbos, ações que os objetos fazem.

```
class Calculadora {  
    public indiceMassaCorporal = (peso, altura) => {  
        return peso / (altura*altura)  
    }  
}
```

Cuidados...

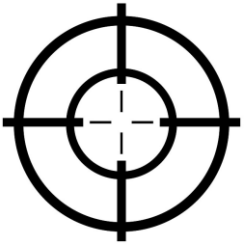


Não use gírias, nem trocadilhos.

Selecione uma palavra por contexto. Cuidado com sinônimos, na hora de compor o nome dos métodos.

Evite usar os nomes associados ao domínio do problema. Como termos ou jargões dos usuários da aplicação.

Métodos ou funções



```
const processar = (dados) => {  
  let novos = []  
  for (let i = 0; i < dados.length; i++) {  
    let dado = dados[i].toUpperCase()  
    novos.push(dado)  
  }  
  novos.sort()  
  novos.reverse()  
  let filtrados = novos.filter(function(elemento){  
    return !elemento.startsWith("A")  
  })  
  return filtrados  
}
```

O que método faz? Tente responder, rapidamente.

Métodos ou funções



Faça os métodos pequenos. Métodos grandes são confusos e dificultam a manutenção ou evolução do código.

Regra de ouro: métodos (funções), devem fazer uma coisa, devem fazê-la bem, devem fazer apenas ela.

Métodos ou funções



```
const colocarEmMaiusculo = (dados) => {  
  let maiusculos = []  
  for (const dado of dados) {  
    maiusculos.push(dado.toUpperCase())  
  }  
  return maiusculos  
}
```

```
const colocarOrdemReversa = (dados) => {  
  dados.sort()  
  dados.reverse()  
}
```

```
const processar = (dados) => {  
  let processados = colocarEmMaiusculo(dados)  
  colocarOrdemReversa(processados)  
  processados = filtrar(processados)  
  return processados  
}
```

```
const filtrar = (dados) => {  
  let filtrados = dados.filter(function (dado) {  
    return !dado.startsWith("A")  
  })  
  return filtrados  
}
```

Métodos ou funções



Métodos pequenos são mais fáceis de entender. Por isso são mais fáceis de atualizar.

Partes pequenas podem ser reaproveitadas, sem necessariamente ocorrer duplicação de código.

Importante! A quantidade de métodos em uma classe também deve ser reduzida. Considere o contexto da classe, para o qual ela foi criada.

Comentários



As vezes não há sucesso na clareza da codificação. Nestas situações será necessário comentar o código.

Um código bem feito não precisa de comentários. Antes de comentar, verifique primeiro se não é possível melhorar o código.

Comentários são a única forma de salvar o entendimento de um código mal feito. Então, se vai comentar faça isso com clareza.

Comentários não são feitos apenas para entender a execução do código, servem para avisar sobre outras coisas como correções, por exemplo.

SOLID

Em engenharia de software, SOLID é um acrônimo para cinco princípios de design que visam tornar os designs de software mais compreensíveis, flexíveis e fáceis de manter.

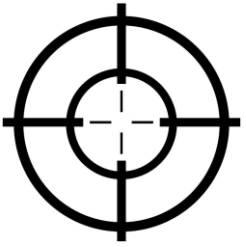
Os princípios são: responsabilidade única, aberto-fechado, substituição de Liskov, segregação de interface, inversão de dependência.

Responsabilidade única



O princípio de responsabilidade única afirma que cada módulo do software deve ter um e apenas um motivo para mudar.

Responsabilidade única



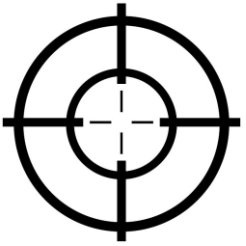
Contexto: imagine que você trabalha em e-commerce e encontrou a seguinte classe já desenvolvida.

```
class Produto {  
    public nome: string  
    public lote: string  
    public fabricante: string  
    public dataFabricacao: string  
    public preco: number  
    public descricao() {  
        return `Nome: ${this.nome}\n${this.lote}`  
    }  
    public calcularPrecoFinal(taxa: number) {  
        return this.preco * taxa  
    }  
}
```

A descrição de um produto, pode mudar? Em outras palavras, é impossível alguém pedir uma atualização neste método?

O mesmo pode acontecer com o cálculo do valor final. Seria impossível alguém pedir mudança neste método?

Responsabilidade única



```
class Produto {  
    public nome: string  
    public lote: string  
    public fabricante: string  
    public dataFabricacao: string  
    public preco: number  
}
```

```
const calculadorPreco = (produto: Produto, taxa: number) => {  
    return produto.preco * taxa  
}
```

```
const descriptor = (produto: Produto) => {  
    return `Nome: ${produto.nome}\n${produto.lote}`  
}
```

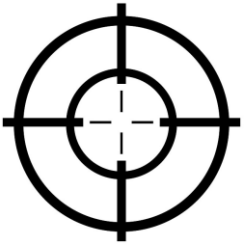
Retira-se da classe
toda responsabilidade
que ela não deve ter!

Aberto-fechado



Na programação orientada a objetos, o princípio aberto-fechado declara “entidades de software (classes, módulos, funções, etc.) devem ser abertas para extensão, mas fechadas para modificação”.

Aberto-fechado

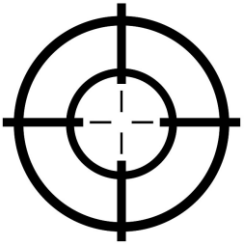


Contexto: imagine que você trabalha em e-commerce e encontrou a seguinte classe já desenvolvida.

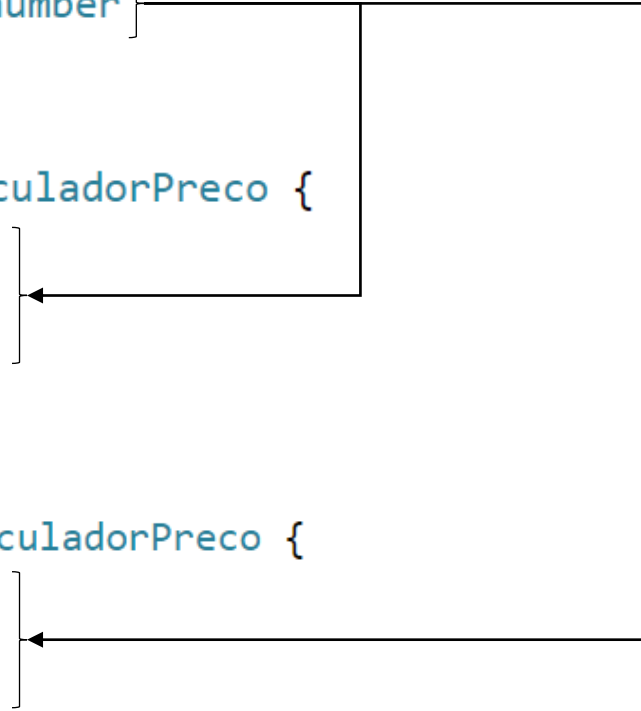
```
class CalculadorPreco {  
    public produto: Produto  
    public calcularPreco = (taxa: number) => {  
        return this.produto.preco * taxa  
    }  
}
```

Seria possível inserir um novo tipo de desconto, sem alterar o código já feito pela classe?

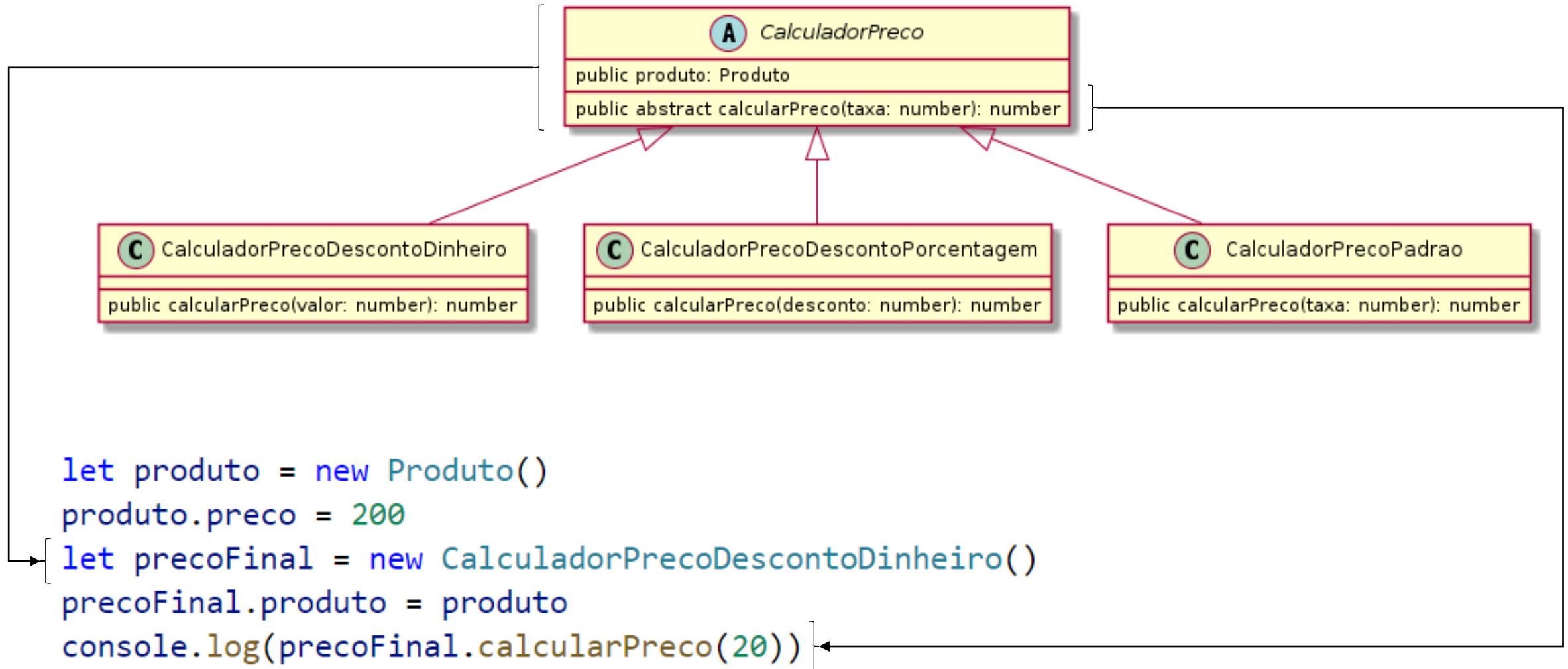
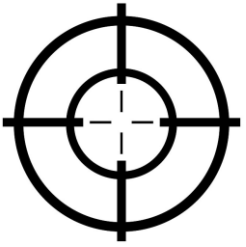
Aberto-fechado



```
abstract class CalculadorPreco {  
    public produto: Produto  
    public abstract calcularPreco(taxa: number): number  
}  
  
class CalculadorPrecoDescontoDinheiro extends CalculadorPreco {  
    public calcularPreco(valor: number): number {  
        return this.produto.preco - valor  
    }  
}  
  
class CalculadorPrecoDescontoPorcentagem extends CalculadorPreco {  
    public calcularPreco(desconto: number): number {  
        let preco = this.produto.preco  
        return preco - (preco * (desconto / 100))  
    }  
}
```



Aberto-fechado

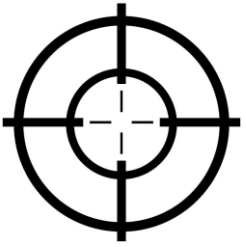


Substituição de Liskov



O Princípio de substituição de Liskov define que os objetos de uma superclasse devem ser substituídos por objetos de suas subclasses sem “quebrar” a aplicação, ou seja, sua execução.

Substituição de Liskov



```
let produto = new Produto()  
produto.preco = 200  
let precoFinal = new CalculadorPrecoDescontoPorcentagem()  
precoFinal.produto = produto  
console.log(precoFinal.calcularPreco(20))
```

```
let produto = new Produto()  
produto.preco = 200  
let precoFinal = new CalculadorPrecoDescontoDinheiro()  
precoFinal.produto = produto  
console.log(precoFinal.calcularPreco(20))
```

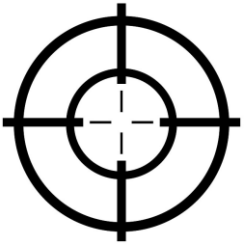
Para aplicação, faz diferença o tipo de cálculo de desconto? O produto sabe o tipo de desconto que está sendo aplicado?

Segregação de interface



Este princípio afirma que nenhum cliente deve ser forçado a depender de métodos que não utiliza.

Segregação de interface

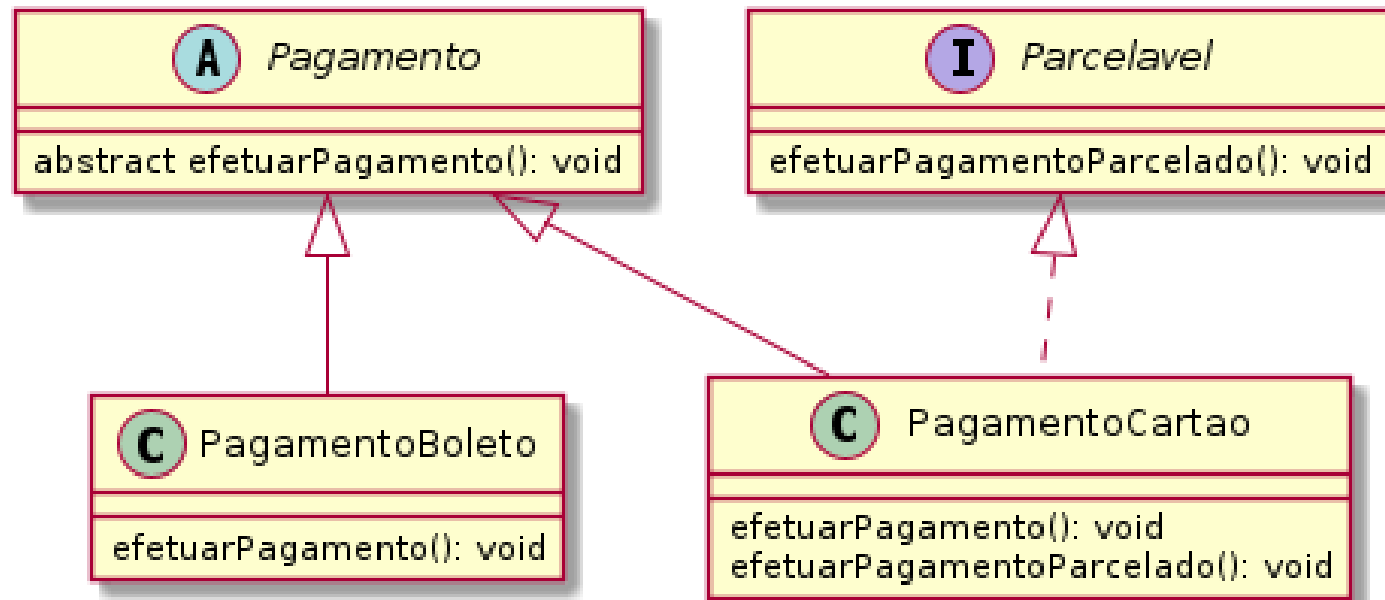
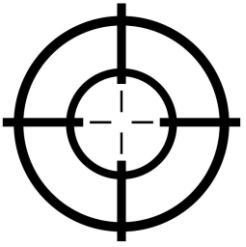


```
abstract class Pagamento {  
    abstract efetuarPagamento(): void  
    abstract efetuarPagamentoParcelado(): void  
}  
  
class PagamentoBoleto extends Pagamento {  
    efetuarPagamento(): void {}  
    efetuarPagamentoParcelado(): void {}  
}
```

A diagram line connects the closing brace of the abstract method `efetuarPagamentoParcelado(): void` in the `Pagamento` class to the corresponding method implementation in the `PagamentoBoleto` class, illustrating that the concrete class implements the abstract method.

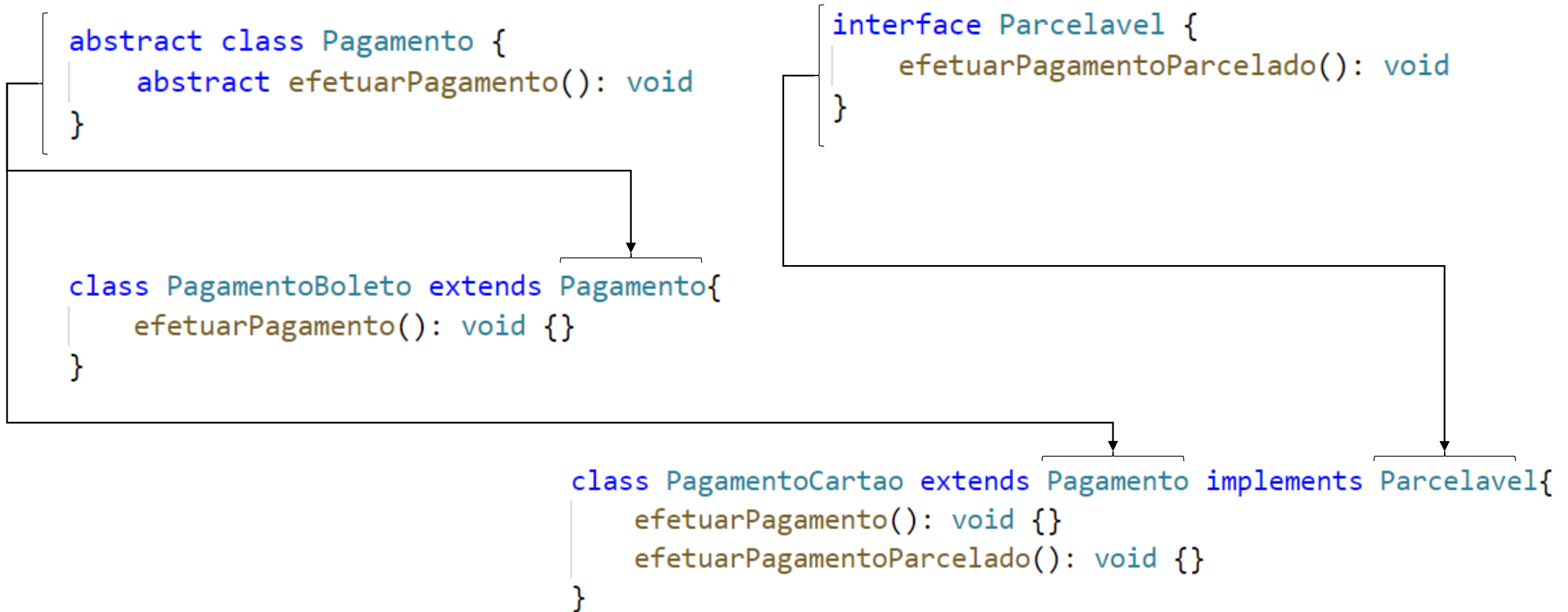
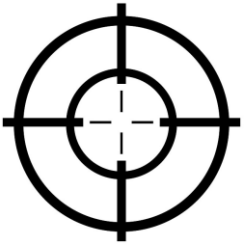
Faz sentido, pagamento por boleto ter um método de parcelamento?

Segregação de interface



Faz mais sentido “colocar” comportamentos numa classe se e somente se ela realmente for utilizá-los em alguma situação.

Segregação de interface



Inversão de dependência



O princípio de inversão de dependência é uma forma de especificar módulos de acoplamento fraco.

Neste princípio, duas premissas são importantes: módulos de alto nível não devem depender de módulos de baixo nível. Ambos devem depender de abstrações (por exemplo, interfaces). Abstrações não devem depender de detalhes. Detalhes (implementações concretas) devem depender de abstrações.

Inversão de dependência

